

GIT & GITHUB — CHEAT SHEET FOR DUMMIES

itc-hybrid-prj · keep next to keyboard · v1.0

The Mental Model (read once)

Git = a save-game system for the whole project folder. GitHub = the online backup of it.

Word	Means
Repository (repo)	The project folder + its entire history.
Commit	A save point with a note attached. You decide when.
Branch	A photocopy of the blueprints you can safely scribble on. main = the official original.
Merge	Combining an approved photocopy back into the original.
Push / Pull	Push = upload your saves to GitHub. Pull = download the latest from GitHub.
Stage (add)	Putting changes in the box before sealing it. Commit seals the box.

One-Time Setup (repo bootstrap)

Step	Command	What it does
1	<code>git init</code>	Turns the current folder into a Git repo. Run inside itc-hybrid-prj.
2	<code>git add .</code>	Stages everything in the folder.
3	<code>git commit -m "message"</code>	First save point.
4	<code>git branch -M main</code>	Names the main branch 'main'.
5	<code>git remote add origin <URL></code>	Connects the local repo to the empty GitHub repo (URL from GitHub's page).
6	<code>git push -u origin main</code>	First upload. After this, plain 'git push' is enough.

Also once per machine, if never done: tell Git who you are:

Command	
<code>git config --global user.name "Ioannis"</code>	Your name on every commit
<code>git config --global user.email "you@email"</code>	Use the email linked to GitHub

The Daily Rhythm (one task, start to finish)

Step	Command	Plain words
1. Get current	<code>git pull</code>	Download latest from GitHub before starting. Habit, even solo.
2. New branch	<code>git checkout -b feature/task-name</code>	Create the photocopy and switch to it, in one go.
3. Work	(edit files in Obsidian / editor)	Do the actual task.
4. Check	<code>git status</code>	See what changed. Run it often — it is your best friend.
5. Stage	<code>git add .</code>	Put all changes in the box. (Or name

Step	Command	Plain words
		one file instead of the dot.)
6. Save	<code>git commit -m "Task: ... " -m "Status: ..." -m "Summary: ..."</code>	Seal the box with our 3-line standard (each -m = one line).
7. Upload	<code>git push -u origin feature/task-name</code>	Branch now visible on GitHub too.
8. Go home	<code>git checkout main</code>	Switch back to the original.
9. Merge	<code>git merge feature/task-name</code>	Approved work becomes official.
10. Clean up	<code>git branch -d feature/task-name</code>	Throw away the photocopy.
11. Upload main	<code>git push</code>	GitHub gets the final state.

Remember: board updates (kanban-board.md) happen on main only — steps 1, then move card, add, commit, push. Never on a feature branch.

Looking Around

Command	Shows you
<code>git status</code>	What changed, what is staged, which branch you are on.
<code>git log --oneline</code>	History, one save point per line.
<code>git branch</code>	All branches; * marks where you are.
<code>git diff</code>	Exact changes not yet staged, line by line.

When Things Go Wrong (they will — it's fine)

Situation	Fix	Notes
Typo in the last commit message	<code>git commit --amend -m "new message"</code>	Only if not pushed yet.
Staged a file by mistake	<code>git restore --staged filename</code>	Unstages; your edits stay.
Ruined a file, want the saved version back	<code>git restore filename</code>	CAREFUL: your local edits to it are gone.
Committed on main instead of a branch	<code>git branch feature/task-name --</code> <code>then git reset --hard</code> <code>origin/main</code>	Branch keeps the work; main goes back to match GitHub.
Push rejected ('fetch first')	<code>git pull</code> , then <code>git push</code>	GitHub had newer commits than you.
Total confusion	<code>git status</code> and read it slowly	It literally tells you what to do next.

The Golden Rules

#	Rule
1	One task = one branch. One branch = one deliverable.
2	Commit small and often. A save point costs nothing; lost work costs an evening.
3	Never work directly on main (only exception: moving Kanban cards).
4	<code>git status</code> before and after everything, until it's instinct.
5	Push at the end of every session — GitHub is your off-site backup.
6	Commit messages follow the standard: Task / Status / Summary. Future-you reads them.